BT1

```
interface Mail {
    void sendMail();
    void log();
}
class MailImpl implements Mail {
    @Override
    public void sendMail() {
        System.out.println("Dang gui mail"
    }
    @Override
    public void log()
        System.out.println("Dang ghi log")
    }
}
public class Main {
    public static void main(String[] args) {
        MailImpl m = new MailImpl();
        m.sendMail();
        m.log();
    }
}
```

HÒA BÌNH

Được quét bằng CamScanner

BT2

```
interface Shape {
    void draw ();   String color = "red";
    ~~public void getArea ()~~
    ~~public void getPerimeter ()~~
}
class Rectangle implements Shape {
    @ Override
    public void draw () {
        System.out.println("vẽ hcn " + color
                                ~~+ getArea + getPerimeter ()~~);
    }

class Circle implement Shape {
    @ Override
    public void draw () {
        System.out.println("draw" + color + "cirle");
    }
}
```

HÒA BÌNH

Được quét bằng CamScanner

BT4

```
abstract class Car {
    protected String color;
    protected double speed;
    public Car (String color, double speed ){
        this. color = color;
        this speed = speed;
    }
    public String getColor (){
        return color;
    }
    public double getSpeed (){
        return speed;
    }
    public abstract void accelerate ();
}
class Bmw extend Car{
    public Bmw (String color, double speed) {
        super (color, speed);
    }
    @ Override
    public void accelerate () {
        speed += 20;
    }
}
```

HÒA BÌNH

Được quét bằng CamScanner

```
    @Override
    public String toString() {
        return "bmw" + color + speed;
    }
}
class Civic extend Car {
    public Civic (String color, double speed) {
        super (color, speed);
    }
    @Override
    public void accelerate() {
        speed += 15;
    }
    @Override
    public String toString() {
        return "Civic" + color + speed;
    }
}
```

HÒA BÌNH

Được quét bằng CamScanner

```
public class Main {
    public static void main (String [] args) {
        Bmw c1 = new Bmw ("Red", 100);
        Civic c2 = new Civic ("Blue", 80);
        c1. accelerate ();
        c2. accelerate ();
    }
}
BT5:
interface Moveable {
    void moveUp ();
    void moveDown ();
    void moveLeft ();
    void moveRight ();
}
class MovableCircle implements Moveable {
    @Override
    public int radius
    MovablePoint center;
    public MovableCircle (MovablePoint center, int radius
        this. center = center;
        this. radius = radius;
    }
```

HÒA BÌNH

Được quét bằng CamScanner

```
    public void moveUp() {
        center.moveUp();
    }
    public void moveDown() {
        center.moveDown();
    }
    public void moveLeft() {
        center.moveLeft();
    }
    public void moveRight() {
        center.moveRight();
    }
    public String toString() {
        return "Circle center " + center + "radius" + radius;
    }
}
class MovablePoint implements Movable {
    int x;
    int y;
    int xSpeed;
    int ySpeed;
```

HÒA BÌNH

Được quét bằng CamScanner

```
public Movable (int x, int y, int xSpeed,
                                int ySpeed){
    this.x = x;
    this.y = y;
    this.xSpeed = xSpeed;
    this.ySpeed = ySpeed;
}
public void moveUp(){
    y += ySpeed;
}
public void moveDown(){
    y -= ySpeed;
}
public void moveLeft(){
    x -= xSpeed;
}
public String toString(){
    return "Point("+x+","+y+")";
}
}
```

HÒA BÌNH

Được quét bằng CamScanner

```
public class Main {
    public static void main (String[] arg) {
        MovablePoint p = new MovablePoint (0,0,2,2);
        ________ Circle c = ____________ Circle (p, 5);
        System.out.println (c);
        c.moveRight ( );
        c.moveUp ( );
    }
}
```

HÒA BÌNH

Được quét bằng CamScanner